

Session 1: Quality Fundamentals and the Tester Mindset

Chapter 1: Why is testing necessary? (QA Mission and Objectives).....	2
1. The Need for Testing: Human and Software Complexity	3
2. Fundamental Objectives of Testing	3
3. The Economic Impact of Testing (Cost of Failure)	3
💡 Practical Examples for Discussion	4
Did you know that...?	4
Applied Story: "Seat Belt"	4
🧑‍🔬 Group activity: "The Tester's Eye".....	5
Chapter 2: The Quality Chain (Error vs. Defect vs. Failure)	5
1. Error (Error / Mistake).....	5
2. The Defect (Defect / Bug / Fault)	6
3. Failure.....	6
🔗 Chain of Causation: How does the error propagate?	6
💡 In-depth analysis: Why does the distinction matter?	7
Did you know that...?	7
Applied Story: "The Cake Recipe"	7
🧑‍🔬 Thinking Exercise: "Identify the Chain".....	8
Chapter 3: The Psychology of Testing and Levels of Independence	9
1. Tester Mindset vs. Developer Mindset.....	9
2. Communication and Diplomacy in QA	10
3. Testing Independence Levels (ISTQB Standard).....	10
4. The 7 Principles of Testing (Synthesis from Individual Study)	11
💡 Practical Examples for Discussion	11
Did you know that...?	12

Applied Story: "The Writer and the Proofreader"	12
🧐 Thinking Exercise: "Curiosity Challenge"	12
Chapter 4: Risk Management in Testing (Product vs. Project)	13
1. What is Risk?.....	13
2. Product Risk	13
3. Project Risk.....	14
📊 Testing Prioritization Matrix	14
💡 Practical Examples for Discussion	14
Did you know that...?	15
Applied Story: "The Bridge Over the River"	15
🧐 Thinking Exercise: "Identification Session"	16
Bonus Chapter: Testing vs. Debugging and the Role of Quality Advocate	16
1. Testing vs. Debugging: Who, What and When?.....	17
A. Testing (QA Activity)	17
B. Debugging (Developer Activity).....	17
📊 Collaboration Flow: Quality Circle	17
2. The Tester as a "Quality Advocate"	18
💡 Practical Examples for Discussion	18
Did you know that...?	18
Applied Story: "The Doctor vs. the Surgeon"	19
🧐 The "Interview" Challenge: Testing vs Debugging.....	19
Chapter 5: Recap, Conclusions and Practical Mission	20
1. Comprehensive Recap: What did I put in the "toolkit"?	21
2. Session Conclusions: QualiAdept's "Golden Laws"	21
3. Homework: "QA Detective Mission" 🚀	22
Part A: Failure Chain Analysis (Mapping).....	22
💡 Final tip from the QualiAdept team.....	23

Chapter 1: Why is testing necessary? (QA Mission and Objectives)

 In this chapter, we don't limit ourselves to dry definitions. We explore the reason for the existence of a QA department and the impact that a tester has on the success (or failure) of a company. Testing is not an optional activity at the end of the project; it is "**the belt**" of **safety**" of the entire development process.

1. The Need for Testing: Human and Software Complexity

Software is created by humans, for humans. For this reason, it is inherently prone to imperfection.

- **Human Limits:** No matter how good a programmer is, fatigue, time pressure, or simple misunderstanding of a business requirement can lead to mistakes.
- **Systems Complexity:** Modern applications are interconnected networks. A minor change in one module can cause a "domino effect" and break completely different functionality.
- **Variable Environment:** Software runs on thousands of different types of devices, browsers, and networks. What works on a developer's laptop may fail miserably on a customer's phone on another part of the world.

2. Fundamental Objectives of Testing

Contrary to the myth that testing means "just finding bugs", our goals are much more strategic:

- **Product rating:** We verify whether what was built corresponds to what was requested (Business Requirements).
- **Defect prevention:** By involving QA early in the design phase, we can identify ambiguities in requirements before the first line of code is written (Early Testing).
- **Risk reduction:** We cannot guarantee that the software has 0 bugs, but we can reduce the risk of a critical defect reaching the customer.
- **Gaining trust:** We provide stakeholders (managers, customers) with a clear and honest picture of the quality level. If the tester says "it's done", the customer can sleep peacefully.
- **Compliance:** In fields such as medical, banking, or automotive, testing is mandatory by law to ensure user safety.

3. The Economic Impact of Testing (Cost of Failure)

There is a direct correlation between when a bug is found and the cost of fixing it:

- **Found in the Analysis phase:** It costs €1 (we just modify a text in a document).

- **Found in the Programming phase:** It costs €10 (the programmer corrects his code on the spot).
- **Found in QA phase:** Costs €100 (requires reporting, re-testing, management time).
- **Found by Customer (Production):** It can cost millions of euros (image damage, legal proceedings, loss of customers).

💡 Practical Examples for Discussion

Scenario A: The 10 Cent Error

- **Context:** A billing system incorrectly rounds each transaction by 10 cents.
- **Question:** Does this seem like a minor problem?
- **QA Analysis:** If the bank processes 10 million transactions per day, the "only 10 bani" error becomes a loss of 1 million euros daily. Testing not only checks whether the button is blue, but also the mathematical logic behind it.

Scenario B: Glovo/Uber app at peak hours

- **Context:** The app works perfectly at 10 a.m. At 7 p.m., when thousands of people are ordering simultaneously, the system crashes.
- **QA Analysis:** This is a problem of **Performance** Testing must predict the "stress" the product will be subjected to in real life.

Did you know that...?

💡 In 1996, the Ariane 5 rocket exploded 37 seconds after launch due to a data conversion software error (a 64-bit number was forced into a 16-bit space)? The loss was **370 million dollars**. A simple unit test could have prevented this catastrophe.

Applied Story: "Seat Belt"

🔧 Imagine you are the safety engineer of a car factory.

- **Programmers** - They are the mechanics who assemble the engine and the body. They want the car to be fast and beautiful.
- **Tester** - is the one who puts the car in the "Crash Test". You don't want to destroy the car

because you are bad, but because you want to know if the airbag deploys **when it matters**. Without you, the car might look perfect in the showroom, but it would be a death trap on the highway. Your mission is not to "break" the car, but to make sure it protects the user's life in the harshest conditions.

Group activity: "The Tester's Eye"

Look at the object next to you (a mug, a pen, your phone).

- What is it? **Function** his main? (**Happy** Path)
- How could it go wrong if used incorrectly? (**Negative** Testing)
- What would happen if 100 people tried to use it simultaneously? (**Load** Testing)

Chapter 2: The Quality Chain (Error vs. Defect vs. Failure)

 In this chapter, we debunk the common confusion between the terms “bug,” “error,” and “problem.” To a typical user, the application “doesn’t work.” To a QualiAdept tester, there is a cause, a manifestation, and a result. Understanding this chain is essential for writing professional bug reports and helping programmers find the source of the problem.

1. Error (Error / Mistake)

 **Definition:** A human action that produces an incorrect result.

- **Source:** The human brain. It's a mistake in judgment, typing, or interpretation.
- **Common causes:**
 - **Time pressure:** "We have to finish by Friday!" (leads to inattention).
 - **Cod complex:** New technologies or mathematical logics that are hard to follow.
 - **Poor communication:** The programmer understood that the button should be "Green",

but the client wanted "Blue".

- **Lack of experience:** Ignorance of the programming language or domain (e.g. a programmer writing medical software without understanding clinical terms).

2. The Defect (Defect / Bug / Fault)

 **Definition:** Static manifestation of an error in software or documentation.

- **Source:** The source code, the specifications, or the design. It's the "mistake" sitting there, waiting to be executed.
- **Where is it found:**
 - In lines of code (e.g. a wrong calculation formula).
 - In business requirements (e.g. a document that says the user can be -5 years old).
 - In the database (e.g. a missing column).
- **Important note:** A defect can exist in the system for decades without ever being discovered, if that portion of code is never run by a user.

3. Failure

 **Definition:** Deviation of the component or system from delivering the expected service.

- **Source:** User interaction with the system during runtime.
- **Manifestation:** What we see on the screen: an error message, a blue screen, a miscalculated amount, or the application that suddenly closes.
- **Relationship with the Defect:** A failure is always caused by a defect. However, a failure can also be caused by external factors (e.g. power outages, internet outages, electromagnetic radiation that corrupts data), without necessarily being a bug in the code.

 **Chain of Causation: How does the error propagate?**

The process follows a strict logical line:

The Man (makes a Mistake) → Insert a Defect (in code) → User runs the code and sees a Failure.

Complete example:

- **Error:** A tired programmer confuses the > (greater than) operator with >= (greater than or equal to) when writing code to check the legal voting age.

- **Defect:** In the source code it remains written: `if (age > 18) { canVote = true; }.`
- **Failure:** A young man of exactly 18 years old enters the site, enters his age, but the system tells him that he does not have the right to vote (although legally he should have).

💡 In-depth analysis: Why does the distinction matter?

As a tester, you don't stop at "it doesn't work." You do **RCA (Root Cause Analysis)**.

- If you only report **Failure** ("I can't vote at 18"), the programmer has to look for the needle in the haystack himself.
- If you report **defect** ("The age condition does not include the value 18"), the programmer fixes the bug in 10 seconds.

Did you know that...?

💡 Don't all defects lead to failures?

Think of a spare tire on a car that has a hole (Defect). As long as you don't get a flat tire and use the tire, the car runs perfectly (you don't have a Failure). But the second you try to use it, your system "fails." In software, this is called **Dead Code** or portions of code that are activated only under rare conditions (e.g. only on February 29th).

Applied Story: "The Cake Recipe"

🧠 Imagine you want to make a cake from a recipe in a book.

1. **Error:** The author of the book was distracted when writing the recipe and wrote "1 kg of salt" instead of "1 teaspoon of salt."
2. **The defect:** In your cookbook (which is "The Code"), there is now a wrong ingredient written. The book is sitting on the shelf, not bothering anyone.
3. **Failure:** You start cooking. You add a kilo of salt, put it in the oven, and in the end you get

an inedible cake.

Question for the student: Where should the tester have intervened to save money on ingredients?

- *Response:*At the stage of **Static Testing**, reading the recipe before starting to cook and noting that 1kg of salt is a **Defect** logic.

Thinking Exercise: "Identify the Chain"

Analyze the following situation and identify the Error, Defect, and Failure:

"An engineer misconfigured the mail server, putting the wrong recipient address in the settings file. Every month, customer invoices go to an unknown person, and the company doesn't get its money."

1. **The error is:** _____
2. **The defect is:** _____
3. **The failure is:** _____

Chapter 3: The Psychology of Testing and Levels of Independence

✚ The success of a testing process depends largely on the human factor. Testing can be perceived as a “negative” activity (looking for defects, criticizing the work of others), but in reality it is a constructive activity, protecting the product. In this chapter, we explore how a QA should think, how they should communicate, and why it is vital that the person testing is different from the person who wrote the code.

1. Tester Mindset vs. Developer Mindset

Although both roles collaborate towards the same goal (delivering a good product), they require distinct and sometimes opposing mindsets.

- **The Developer (Creator) Mindset:**
 - **Scope:** To build functionality.
 - **Vision:** "How can I make this button save data to the database?"
 - **Psychological Barrier:** It's difficult to find your own mistakes. Our brains tend to see what they "think" they wrote, not what they actually wrote (**Confirmation Bias**).

- **The Tester Mindset (Detective):**

- **Scope:** To verify resilience and compliance.
- **Vision:** "What happens if I press the button 100 times?", "What happens if I disconnect the internet while the data is being saved?".
- **Key Features:**
 - **Curiosity:** The desire to explore the hidden corners of the application.
 - **Professional Pessimism:** The assumption that "there's a bug somewhere, I just need to find it."
 - **Critical Eye:** Attention to details that seem insignificant (a shifted pixel, a second of delay).
 - **Attention to details:** Ability to notice discrepancies between requirements and reality.

2. Communication and Diplomacy in QA

A tester who finds 100 bugs but doesn't know how to communicate them will create tension in the team. Reporting a bug is essentially delivering bad news. Therefore, communication must be:

- **Objective, not personal:** We don't say "You're wrong here," but "The system is behaving differently from specifications in this scenario."
- **Based on facts:** The bug report must contain clear steps, evidence (screenshots/logs) and the expected result.
- **Diplomat:** The Tester and the Developer are not enemies. They are like a pilot and a co-pilot. If one makes a mistake, both crash.
- **Solution-oriented, not blame-oriented:** The goal is to repair the defect, not find the culprit.

3. Testing Independence Levels (ISTQB Standard)

Independence increases the effectiveness of testing. The further away the tester is from the person who created the code, the more defects they will find.

- **Level 0 (No independence):** The author tests his code himself. (Fastest, but least efficient – Confirming errors is almost impossible).
- **Level 1:** Testing is done by another person from the same team (e.g. another developer – **Peer Review**).
- **Level 2:** An independent testing team within the same organization (specialized QAs who know the product but did not write the code).

- **Level 3 (Maximum independence):** External testers (consulting or auditing companies). They come with a completely fresh perspective, without preconceptions about “how it should work.”

Advantages of independence: Independent testers see flaws that creators habitually ignore.

Disadvantages of independence: There may be a communication barrier or a delay in the workflow (QA does not know the technical context as well as dev).

4. The 7 Principles of Testing (Synthesis from Individual Study)

The psychology of testing is guided by these golden rules that temper our expectations:

- **Testing shows the presence of defects, not their absence:** You can prove that bugs exist, but you can never prove that “there are none left.”
- **Exhaustive testing is impossible:** You can't test every combination of data. You have to be strategic.
- **Early testing saves time and money:** QA must start with the requirements.
- **Defect grouping:** Typically, 80% of the problems are in 20% of the modules (**Pareto's principle**).
- **The Pesticide Paradox:** If you run the same tests over and over, they won't find new bugs. Tests need to be updated.
- **Testing depends on the context:** A banking app is tested differently than a phone game.
- **"Absence of errors" error:** There's no point in having bug-free software if it's not useful to the user or doesn't meet requirements.

Practical Examples for Discussion

Scenario A: "The Formed Eye"

- **Context:** A programmer works 14 hours on a payment module. He is proud that everything compiles.
- **QA intervention:** The tester comes in and, instead of entering the amount “100,” enters “ABC” or “-50.” The system crashes.
- **Analysis:** The programmer focused on **as it should** to go. The tester focused on **how can it be fooled** the system.

Scenario B: Team Tension

- **Context:** A tester sends an email to everyone (including managers): “Jon's code is full of

terrible errors, the application is a disaster."

- **Question:** What communication principle was violated?
- **Analysis:** He personalized the bug and created a hostile environment. A QualiAdept tester would have reported the bugs individually in Jira, describing the technical impact, not the quality of the colleague.

Did you know that...?

💡 There is a concept called „**Blindness to Change**” (Change blindness)? If you stare at a screen for too long, your brain starts to ignore small differences. That's why testers take regular breaks or switch modules between them. A fresh pair of eyes will always see the bug you've been "ignoring" for the past 2 hours.

Applied Story: "The Writer and the Proofreader"

💡 Imagine a writer who has just finished a 500-page novel.

The writer has read the text 10 times. He "knows" what he is writing in each sentence. When he gets to a page where he accidentally wrote "apple" instead of "blackberries", his brain will automatically read "blackberries", because that is what he meant to write.

The Corrector (Tester) receives the book for the first time. He has no emotional connection to the text. He reads letter by letter. He will immediately see that "apple" does not make sense in the context of a mountain forest.

Moral: No matter how smart the writer (Developer) is, he needs someone with a different mindset (Tester) to ensure that the message (Application) reaches the reader (Client) without mistakes that ruin their experience.

🧐 Thinking Exercise: "Curiosity Challenge"

We have an age input field on a website. Most people type "25", "30", "45".

Write a list of 5 "strange values" that a curious tester would enter to see if the system holds up. (Ex: 0, 150, "twenty", blank space, -1).

Explain in class: Why did I choose these values? (**Response:** Testing limits and data types).

Chapter 4: Risk Management in Testing (Product vs. Project)

🎯 In an ideal world, we would have infinite time to test everything. In the real world, we have deadlines, fixed budgets, and limited resources. That's why testing is based on **Risk**. A QualiAdept tester does not test randomly; he tests the areas where the probability of failure is highest and where the impact of that failure would be most painful for the business.

1. What is Risk?

Definition: A future, uncertain event that has a negative impact on objectives.

Risk is defined by two essential parameters:

- **Likelihood/Probability:** How likely is it that the problem will occur?
- **Impact:** How much will it hurt if the problem arises?

Risk Formula: $\text{Risk} = \text{Probability} * \text{Impact}$

As QA, our mission is to identify these risks early and use testing as a method of **mitigate** (reduction) of them.

2. Product Risk

This refers to the possibility that the software will not meet user expectations or business requirements. It is the risk related to "what we deliver".

Examples of Product Risks:

- **Functional faults:** The software does not do what it should (e.g. the "Complete Order" button does not respond).
- **Low non-functional quality:** The application is too slow (Performance), difficult to understand (Usability), or vulnerable to hackers (Security).
- **Inability to meet requirements:** The product technically works, but it doesn't solve the

customer's need.

- **Defects that lead to failure:** The presence of critical bugs that block the use of the system.

QA role: We identify product risks through requirements analysis and intensive testing on critical areas (e.g. payment processing in an online store).

3. Project Risk

This refers to external factors or internal processes that may prevent the team from delivering the product on time or within budget. It is the risk related to “how we work.”

Examples of Project Risks:

- **Organizational factors:** Lack of staff (QA goes on leave or resigns), lack of training (the team doesn't know how to use Jira).
- **Technical infrastructure issues:** The testing environment is not ready, the servers are always down, we don't have access to the database.
- **Management/supplier issues:** An external partner does not deliver the API on time, requirements change every day (Scope Creep).

QA role: Although we don't directly control project risks, we need to report them. If the test environment goes down daily, that's a project risk that will delay product testing.

Testing Prioritization Matrix

A smart tester uses a matrix to decide what to test first:

Probability / Impact	Low Impact	High Impact
High Probability	We test if we have time.	MAXIMUM PRIORITY (We test immediately)
Low Probability	We ignore / Test at the end	We test carefully

Practical Examples for Discussion

Scenario A: Banking Application (Product Risk)

- **Risk:** There is a chance that the money transfer will be processed twice.
- **Analysis:** The probability is average (depends on the code), but **Impact** it is huge (financial losses, lawsuits).
- **QA decision:** This scenario is priority 1. We don't launch the application until we are 200% sure that the transaction is unique.

Scenario B: Christmas Launch (Project Risk)

- **Risk:** The online store is supposed to launch on December 1st, but the design team is 2 weeks late.
- **Analysis:** This is a project risk. Testing time is compressed.
- **QA decision:** QA must warn management that reducing testing time increases the risk of critical bugs reaching the customer.

Did you know that...?

💡 During the planning phase, testers participate in training sessions **Risk-Based Testing (RBT)**? In these meetings, the entire team brings up “what scares us the most.” The areas that scare stakeholders the most are the ones that will receive the most test scenarios.

Applied Story: "The Bridge Over the River"

💡 Imagine you are the safety inspector of a newly constructed bridge.

- **Product Risk:** You ask yourself: "Are the cables thick enough?", "Are the pillars earthquake-proof?", "Does the asphalt crack when frozen?". These are related to the construction itself. If the pillar fails, the bridge falls (Critical Failure).
- **Project Risk:** You ask yourself: “Do we have enough workers to finish before the fall floods?”, “Is the steel mill delivering the material on time?”, “Do we have the budget to pay for the crane?”. If the budget runs out, the bridge will never be finished.

QA conclusion: For the user who wants to cross (the Customer), it doesn't matter why the bridge isn't safe or ready. They want a functional bridge. Your mission is to monitor both types of risks to ensure that, in the end, the river crossing is done safely.

Thinking Exercise: "Identification Session"

We have a project to launch a food delivery app (similar to Glovo). Identify and classify the following situations:

- The server we are testing on is very old and restarts itself. (**Risk of:** _____)
- If a customer orders food worth 500 RON, the app incorrectly applies the 10% discount. (**Risk of:** _____)
- The only tester on the team doesn't know how to use the mobile app, only the web app. (**Risk of:** _____)
- When the phone switches from 4G to Wi-Fi, the command is lost. (**Risk of:** _____)

Bonus Chapter: Testing vs. Debugging and the

Role of Quality Advocate

Often, people outside the industry (and even some beginners) confuse testing with fixing code. “If you’re a tester, you fix bugs, right?” – that’s the classic question. In this bonus chapter, we’ll clearly delineate responsibilities and explore the ethical dimension of the QA profession.

1. Testing vs. Debugging: Who, What and When?

Although they are closely related activities, they have different purposes and performers. It is crucial to understand where the tester's work ends and the developer's begins.

A. Testing (QA Activity)

- **Scope:** Identifying failures and verifying compliance with requirements.
- **Process:** The tester runs the software, observes its behavior, and reports discrepancies.
- **Result:** A bug report describing **This** does not work and under what conditions.
- **Mindset:** "I demonstrate that the system is not behaving as it should."

B. Debugging (Developer Activity)

- **Scope:** Identifying the root cause of a defect, isolating it and eliminating it (repairing it).
- **Process:** The programmer analyzes the code, uses special tools (debuggers), monitors system logs, and makes changes to the source files to correct the error.
- **Result:** Corrected code and a new version of the application ready for re-testing.
- **Mindset:** "I find the technical reason why the error occurred and fix it."

Collaboration Flow: Quality Circle

- **Tester** finds a failure (Failure).
- **Tester** write a Bug Report (identify the Defect).
- **Developer** takes the report and begins the process **Debugging** to find the technical source.
- **Developer** fix the code.
- **Tester** re-test (Confirmation Testing) to ensure that the defect has disappeared and no new ones have appeared.

2. The Tester as a “Quality Advocate”

An advanced concept in the QualiAdept model is that the tester is not just a "bug finder", but the voice of the customer within the team.

- **Professional Ethics:** The tester has a moral responsibility to report any risks, even when there is pressure from management to release the product faster.
- **The courage to say "No":** Sometimes, QA is the one who has to say, "This product is not safe for users," thus protecting the company's reputation and customer safety.
- **Empathy towards the user:** The tester must always ask themselves: "If my mother or my child used this app, would they be frustrated? Would they be in danger?"

Practical Examples for Discussion

Scenario A: "Pass the responsibility"

- **Context:** A developer says to the tester: "I fixed the bug, can you go into the code to check if it's written correctly?"
- **QA Analysis:** This is a critical moment. The tester should politely decline to check the source code (if we are doing Blackbox testing) and insist on checking the functional behavior. Debugging and checking the quality of the code at the syntax level is the programmer's job.

Scenario B: "Minor" Bug with Major Impact

- **Context:** There's a bug that causes text on a page to overlap on small screens. The manager says, "It's just visual, leave it like that, we need to launch!"
- **Quality Advocate Perspective:** The tester should explain the impact: "If the text overlaps, the 'Confirm Payment' button becomes inaccessible on 40% of our users' phones. It's not just a visual bug, it's a business blocker."

Did you know that...?

 There is a stage called **Unit Testing** where developers do "testing" on small pieces of code before it gets to QA? Although it's called testing, it's closer to debugging because it's done by the code author to validate their own logic. However, system testing (done by you, the QA) remains the only one that guarantees that all the pieces of the puzzle fit together in

the real world.

Applied Story: "The Doctor vs. the Surgeon"

● Imagine a modern hospital.

- **The tester is the Diagnostician.** You are the one who consults the patient (the Software). You do the analysis, observe the symptoms (the Failures) and make a diagnosis: "The patient has a heart problem" (Bug Report). You do not enter the operation, but you are the one who saved the patient's life because you noticed the problem in time.
- **The developer is the Surgeon.** He takes your diagnosis, opens the patient (Code), looks for exactly where the problem is (Debugging) and makes the necessary intervention to repair the diseased organ.

Moral: The Surgeon (Dev) cannot operate without a correct diagnosis (QA), and the Physician's (QA) diagnosis is worthless if the patient is not operated on (Dev). Both roles are vital, but they are different professions. The Physician (Tester) watches over the patient's overall long-term health, being their "Health Advocate."

The "Interview" Challenge: Testing vs Debugging

If in an interview you are asked: "*Why don't testers fix bugs if they keep finding them?*", what would be your answer now?

- **Ideal answer:** "For reasons of independence and efficiency. A tester has a critical mindset, focused on the whole and the user experience, while fixing requires a constructive mindset and intimate knowledge of the code architecture. Furthermore, if I were to fix the bug myself, I would fall into the 'Confirmation Bias' trap, losing the objectivity necessary to validate the solution later."

Chapter 5: Recap, Conclusions and Practical Mission

💡 A QualiAdept session isn't complete until we make sure the information has moved from "I heard" to "I understand how to apply." This final chapter serves as a compass for everything we've built today: from the economic necessity of testing to the subtle psychology behind

every reported defect.

1. Comprehensive Recap: What did I put in the "toolkit"?

Today we explored five fundamental pillars that define the Manual QA profession:

- **Pillar 1: The Need for Testing.** I understand that software is imperfect because it is a human creation. Testing is not a luxury, but a method of business survival, reducing the enormous costs of defects found too late.
- **Pillar 2: Quality Chain (Terminology).** I have clearly defined the triad:
 - **Error:** The mistake in the mind or hand of man.
 - **The defect (Bug):** The "frozen" error in the code or documentation.
 - **Failure:** The moment the user notices that the system is not working.
- **Pillar 3: Psychology and Independence.** I learned that a tester must be a "professional pessimist" and a "curious detective." I saw why independence (the person testing should be different from the person writing) exponentially increases the chances of finding critical bugs.
- **Pillar 4: Risk Management.** We learned to prioritize. We don't have time to test everything, so we test where the probability is high and the impact is devastating (Product Risk) and monitor factors that can delay delivery (Project Risk).
- **Bonus Pillar: Testing vs. Debugging.** We have delimited the responsibilities. The QA diagnoses (identifies the failure), and the Developer operates (fixes the defect in the code).

2. Session Conclusions: QualiAdept's "Golden Laws"

If you were to leave with just a few ideas after this session, they should be:

- **Testing does not guarantee the absence of bugs.** It only demonstrates the presence of those we were able to find. A "bug-free" software is a software that has not been tested enough.
- **Early testing is the cheapest.** The closer we find the problem to the idea (requirements) phase, the more money and time we save.
- **Communication is as important as technique.** An aggressively reported bug will be rejected; a diplomatically and reasoned bug will be fixed with priority.
- **You are the user's advocate.** Your mission is to ensure that the end customer experience is smooth and secure.

3. Homework: "QA Detective Mission" 🚀

This assignment is designed to train your "tester's eye" on real scenarios. Please write your answers in a document (Word/PDF) and upload them to your personal folder until the next session.

Part A: Failure Chain Analysis (Mapping)

📊 Consider the following scenario:

"An airline is rolling out an update to its check-in system. Due to a misunderstanding between the analyst and the programmer, the system allows passengers to book seats numbered 0, even though the plane starts at row 1. At the boarding gate, 20 passengers are unable to board the plane because their seats don't physically exist."

- Identify **Human error**.
- Identify **The Defect (Bug)** from the system.
- Identify **Failure** noticed by passengers.
- Explain: How could a tester have prevented this in the development phase? **Static Testing** (requirements analysis)?

🧠 Part B: Risk Assessment (Prioritization)

Imagine you are QA for an application like **Mobile Banking** (ex: Revolut/BT Pay in Romania). You need to test two new features, but you only have time for one before launch:

- **Functionality 1:** Changing the background color of the user profile (Personalization).
- **Functionality 2:** Money transfer via phone number (Payments).
- Classify the two functionalities according to **Impact** (Low/High).
- Which of them represents a **Product Risk** bigger if it fails? Why?
- If the test server is down for 3 days, this is a **Product Risk** or of **Design**? Justify.

🎯 Part C: Critical Mindset (Creativity)

Choose a mundane object (e.g. a TV remote control).

- Write 3 type tests **Happy Path** (normal use).
- Write 3 type tests **Negative/Stress** (how would you try to "spoil" or trick her?).

💡 Final tip from the QualiAdept team

Don't be afraid to ask "stupid" questions. In testing, the question "What if we do this...?" is the one that uncovers the most dangerous bugs. See you in Session 2, where we'll learn how to put all these ideas into a **Testing Plan** professional!